

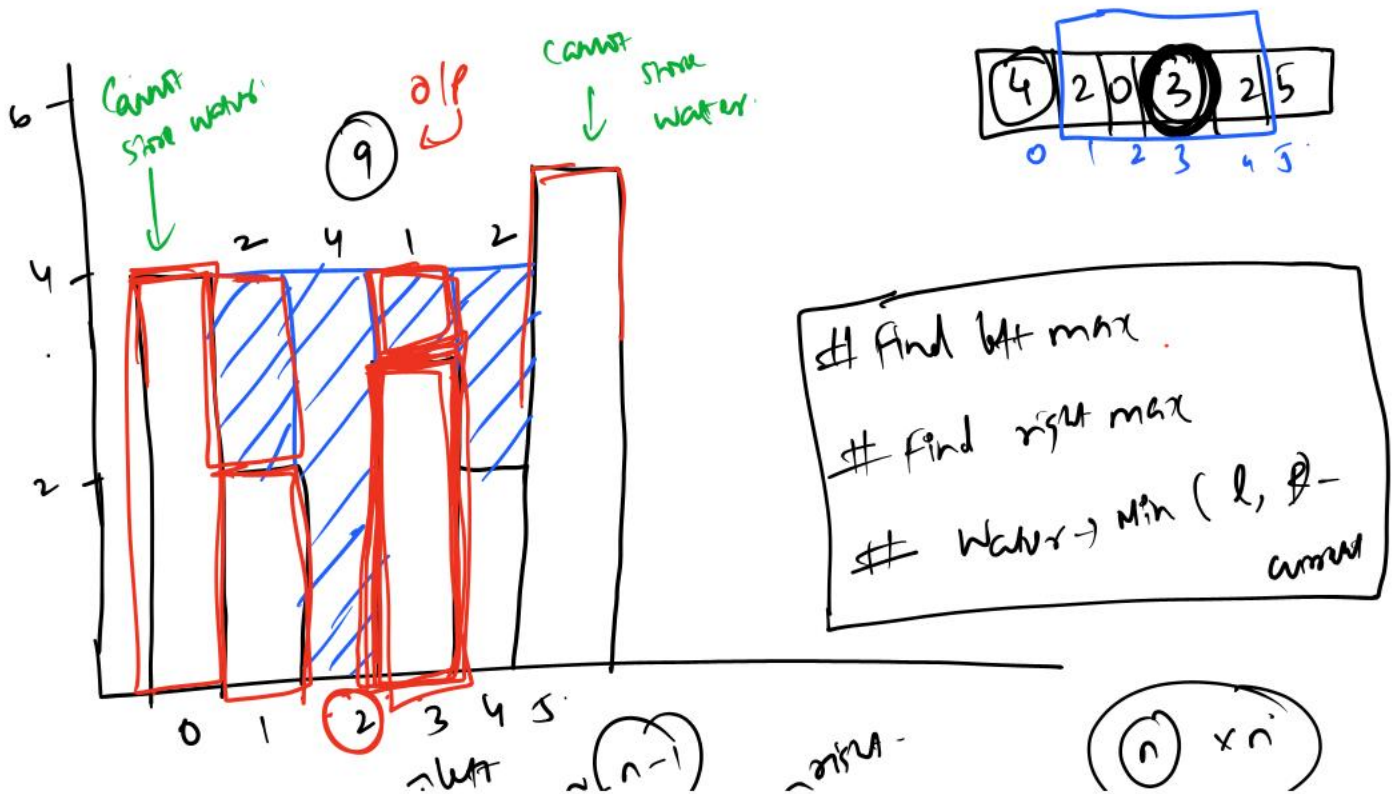
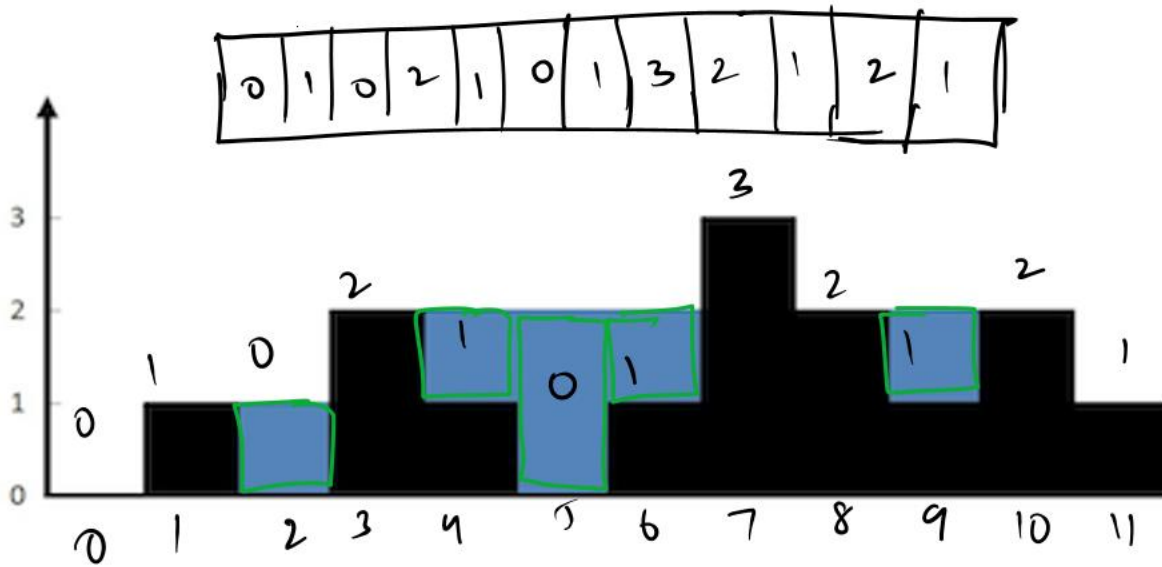
42. Trapping Rain Water

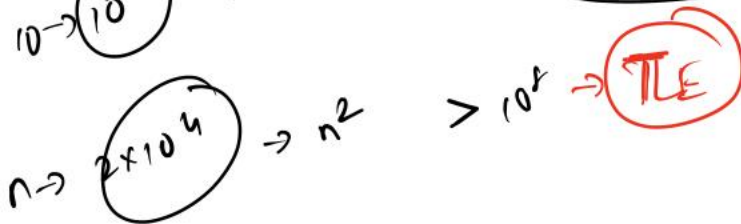
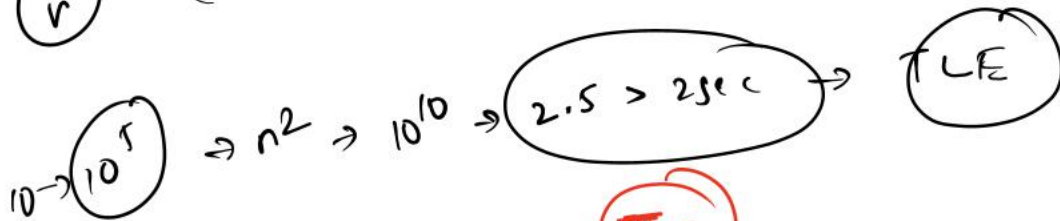
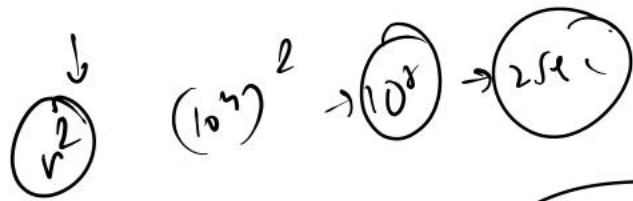
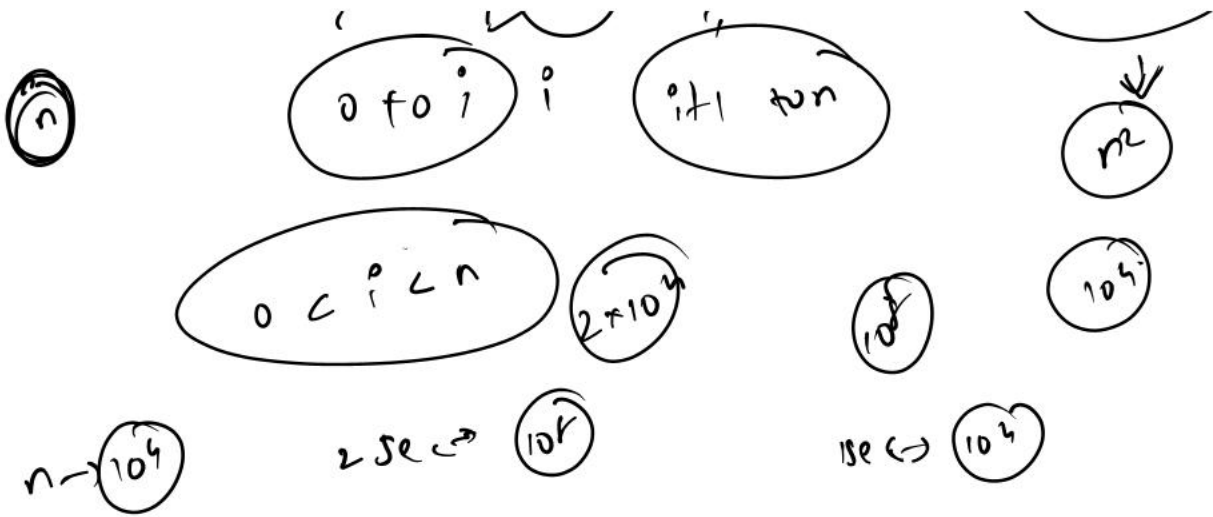
Hard

Topics

Companies

Given n non-negative integers representing an elevation map where the width of each bar is 1 compute how much water it can trap after raining.

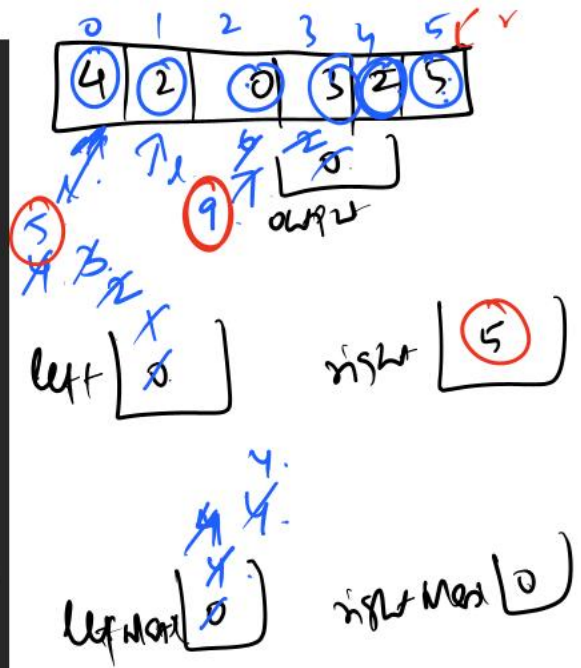




0	1	2	3	4	5
4	2	0	3	2	5

class Solution:

```
def trap(self, height: List[int]) -> int:
    output = 0
    left = 0
    right = len(height) - 1
    leftMax = 0
    rightMax = 0
    while left < right :
        # Find leftmax
        if height[left] < height[right] :
            leftMax = max(leftMax, height[left])
            output += leftMax - height[left]
            left += 1
        else :
            rightMax = max(rightMax, height[right])
            output += rightMax - height[right]
            right -= 1
    return output
```



T.C $\rightarrow O(n)$ S.C $\rightarrow O(1)$

string

125. Valid Palindrome

Solved ✓

Easy

Topics

Companies

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a **palindrome**, or `false` otherwise.

Example 1:

Input: `s = "A man, a plan, a canal: Panama"`

Output: `true`

Explanation: "amanaplanacanalpanama" is a palindrome.

palindrome

"_?A-!-a!?"
↓

A!a

↓
a!a

↓
True

"_ _ _ ? A ! _ _ b _ _ B ! ? * A _ _ _ ?"
↑ ↑ ↑ ↑ ↑

c1 | a)

c2 | a .)

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i = 0
        j = len(s) - 1
        while i < j :
            # Ignore the non alphanumeric characters on left
            while i < j and not s[i].isalnum() :
                i += 1
            # Ignore the non alphanumeric characters on right
            while i < j and not s[j].isalnum() :
                j -= 1
            c1 = s[i].lower()
            c2 = s[j].lower()
            if c1 != c2 :
                return False
            i += 1
            j -= 1
        return True
```

→ P. NO. (125)

→ T.C → $O(n)$

→ S.C → $O(1)$

165. Compare Version Numbers

Solved ✓

Medium

Topics

Companies

Hint

Given two version strings, version1 and version2, compare them. A version string consists of revisions separated by dots ('.'). The value of the revision is its integer conversion ignoring leading zeros.

To compare version strings, compare their revision values in left-to-right order. If one of the version strings has fewer revisions, treat the missing revision values as 0.

Return the following:

- If version1 < version2, return -1.
- If version1 > version2, return 1.
- Otherwise, return 0.

"12.509.73"

"12.509.73.0.0.1"

o/p: (-1)

0 1 2 3
s "0234"

1234
num [1 2 3 4]

0 1 2 3
num [1 2 3 4]

while i < len(s):
num = (num * 10) + int(s[i])
i += 1

0 1 2 3 4 5 6 7
1 2 3 4 5 6 7
↑ ↑ ↑ ↑ ↑
i

25
0 1 2 3
num [1 2 3 4]

0 1 2 3 4 5 6 7 8 9 10 11
1 2 3 4 5 6 7 8 9 10 11
↑ ↑ ↑ ↑ ↑
j

3
0 1 2 3
num2 [1 2 3 4]

0 1
"10"

0 1 2 3
"1002"

3
0 1 2 3
num [1 2 3 4]

10
0 1 2 3
num2 [1 2 3 4]

```
class Solution:
    def compareVersion(self, version1: str, version2: str) -> int:
        i, j = 0, 0
        while i < len(version1) or j < len(version2):
            # Find integral revision of version1 -> till '.' or end of string
            num1 = 0
            while i < len(version1) and version1[i] != '.':
                num1 = (num1 * 10) + int(version1[i])
                i += 1
            # Find integral revision of version2 -> till '.' or end of string
            num2 = 0
            while j < len(version2) and version2[j] != '.':
                num2 = (num2 * 10) + int(version2[j])
                j += 1
            # Compare them
            if num1 < num2:
```

→ P.No: 165

→ T.C → O(m+n)

→ S.C → O(1)

```

    if num1 < num2 :
        return -1
    elif num1 > num2 :
        return 1
    i += 1
    j += 1
    return 0

```

242. Valid Anagram

Solved ✓

Easy

Topics

Companies

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true



"a n a g r a m"

"n a g a r a m"

a	:	x	x	^{0x2}
n	:	x	0	
g	:	x	0	
r	:	x	0	
m	:	x	0	

"abc" |1

"bc" |1

```

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:

```

```

if len(s) != len(t) :
    return False
visited = {}
# Adding the frequency in dictionary
for i in s :
    visited[i] = visited.get(i, 0) + 1
    # if i in visited :
    #     visited[i] = visited.get(i) + 1
    # else :
    #     visited[i] = 1
# Subtract the frequency in dictionary
for i in t :
    if i in visited and visited[i] >= 1:
        visited[i] -= 1
    else :
        return False
return True

```

→ P.No: 242

→ T.C → $O(n)$

→ S.C → $O(1)$

If char is
of alphabet,
no. → S.C.
→ $O(n)$.

dictionary space

746. Min Cost Climbing Stairs

Solved ✓

Easy Topics Companies Hint

You are given an integer array cost where cost[i] is the cost of ith step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

Example 1:

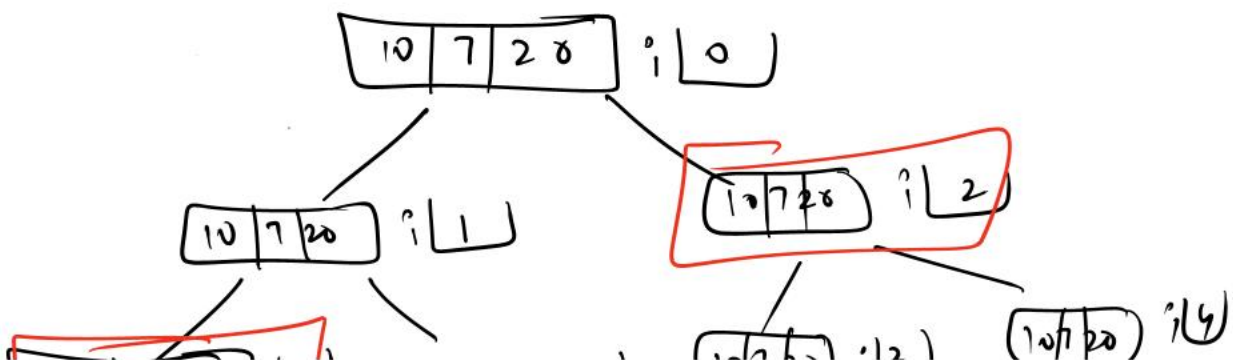
Input: cost = [10,15,20]

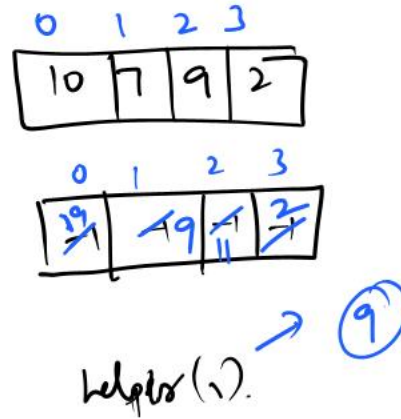
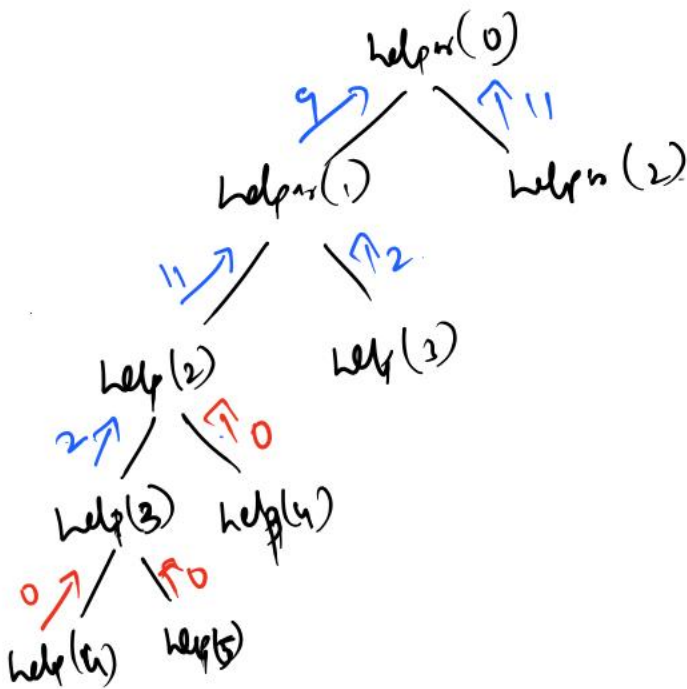
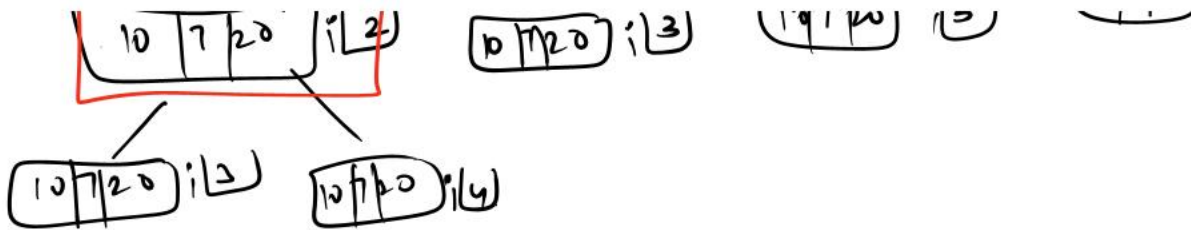
Output: 15

Explanation: You will start at index 1.

– Pay 15 and climb two steps to reach the top.

The total cost is 15.





```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        def helper(i):
            if i >= len(cost):
                return 0
            if dp[i] != -1:
                return dp[i]
            ans1 = helper(i + 1)
            ans2 = helper(i + 2)
            dp[i] = cost[i] + min(ans1, ans2)
            return dp[i]
        dp = [-1] * len(cost)
        return min(helper(0), helper(1))
```

→ P.No

746

→ T.C → $O(n)$

→ S.C → $O(n)$

call stack space
+
dp array space

206. Reverse Linked List

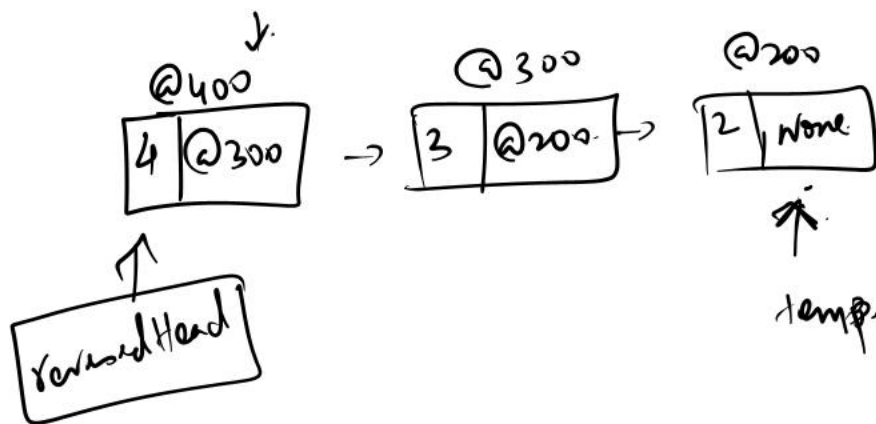
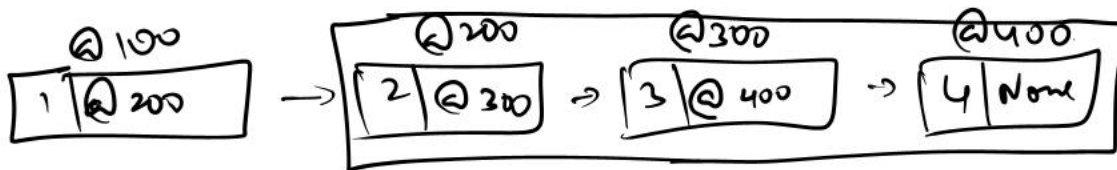
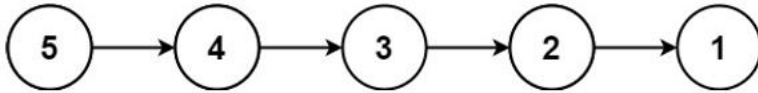
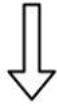
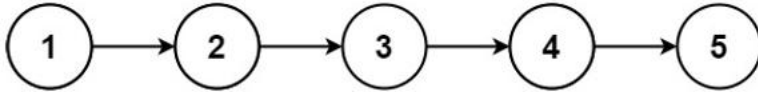
Solved

Easy

Topics

Companies

Given the **head** of a singly linked list, reverse the list, and return the reversed list.

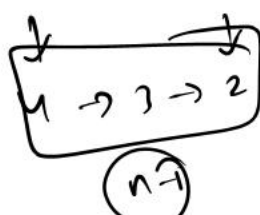


class Solution:

```

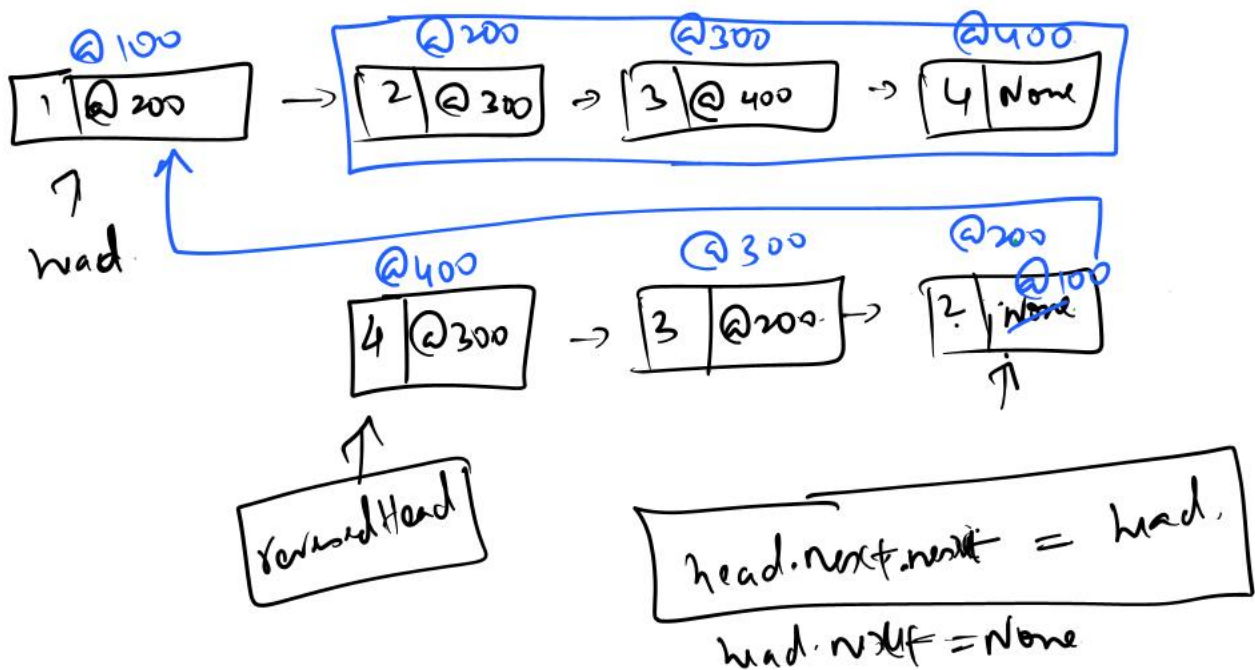
def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
    if head is None or head.next is None :
        return head
    reversedHead = self.reverseList(head.next)
    temp = reversedHead
    while temp.next is not None :
        temp = temp.next
    temp.next = head
    head.next = None
    return reversedHead
  
```

$\Rightarrow T.C \rightarrow O(n^2)$



$n-1 + n-2 + n-3$

$n(n-1) \quad n^2$



```
class Solution:
```

```
def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
```

```
    if head is None or head.next is None :
```

```
        return head
```

```
    reversedHead = self.reverseList(head.next)
```

```
    head.next.next = head
```

```
    head.next = None
```

```
    return reversedHead
```

↓
T.C → $O(n)$

↓
S.C → $O(n)$
↳ call stack space

↓
P.No. (206)